

Baxy

Asistente de IA local, privado y accesible para Windows

Emmanuel Villacura

Portafolio de Proyectos · Ingeniería Civil Informática

19 de junio de 2026

El problema

El dolor no es la ausencia de asistentes, sino su costo, su intromisión y su nula accesibilidad.



Costoso

Exigen 8–12 GB de VRAM o suscripción a la nube. Quedan fuera del alcance del hardware modesto.



Invasivo

Envían tus datos a servidores externos. Sin privacidad real ni control del usuario.



No accesible

Solo texto y teclado. Ignoran a personas con discapacidad visual o motriz.

Objetivos

Objetivo general — Desarrollar un asistente de IA local y multimodal para Windows, operable por voz y texto, que controle el PC priorizando privacidad, bajo consumo de recursos y accesibilidad.

- 1** **Diseñar e implementar** una arquitectura de inferencia que permita ejecutar concurrentemente un LLM, visión y voz dentro de ≤ 4 GB de VRAM, demostrando viabilidad en hardware de gama de entrada.
- 2** **Desarrollar** un mecanismo de tool-calling fiable en hardware modesto, mediante un enrutador especializado (FunctionGemma), que seleccione la acción correcta sin generar herramientas inexistentes.
- 3** **Optimizar** la cadena voz-a-voz para mantener una latencia ≤ 5 segundos por turno, comparable a asistentes comerciales.
- 4** **Garantizar** operación 100% local y privada, sin transmisión de datos a servidores externos, usando exclusivamente software open-source y licencias gratuitas.
- 5** **Implementar** un sistema de honestidad estructural que verifique las acciones contra el estado real del SO, evitando reportar como exitosas acciones que no ocurrieron.
- 6** **Incorporar** modos de accesibilidad por voz (personas no videntes y con movilidad reducida) y soporte multilingüe basado en embeddings semánticos.

Estado del arte / Competencia

Solución	Recursos	Privacidad	Alcance	Accesibilidad
OpenClaw	GPU grande	Local o nube	General · 100+ skills	No
“Jarvis”	Según el modelo	Local (Ollama)	Voz, cámara, skills	No
Mark / AutoGPT	Nube + suscripción	Nube	Variable	No
Baxy	4 GB + opción CPU	100 % local	67 herramientas	Sí · 3 modos

El diferenciador — entre estas alternativas, Baxy es el único pensado para hardware modesto (~4 GB + CPU) con accesibilidad de primera clase: 3 modos WCAG y voz local de punta a punta.

Propuesta de solución: accesibilidad

Una misma base local y privada, con tres modos de accesibilidad alineados a WCAG 2.1 / 2.2.



Normal

Interacción por texto para cualquier usuario, con respuestas habladas.

**Interacción por voz en pruebas
PROXIMAMENTE DISPONIBLE**



No vidente

Narra cada acción en voz alta; navegación y control 100 % auditivos.

**PROXIMAMENTE
DISPONIBLE**

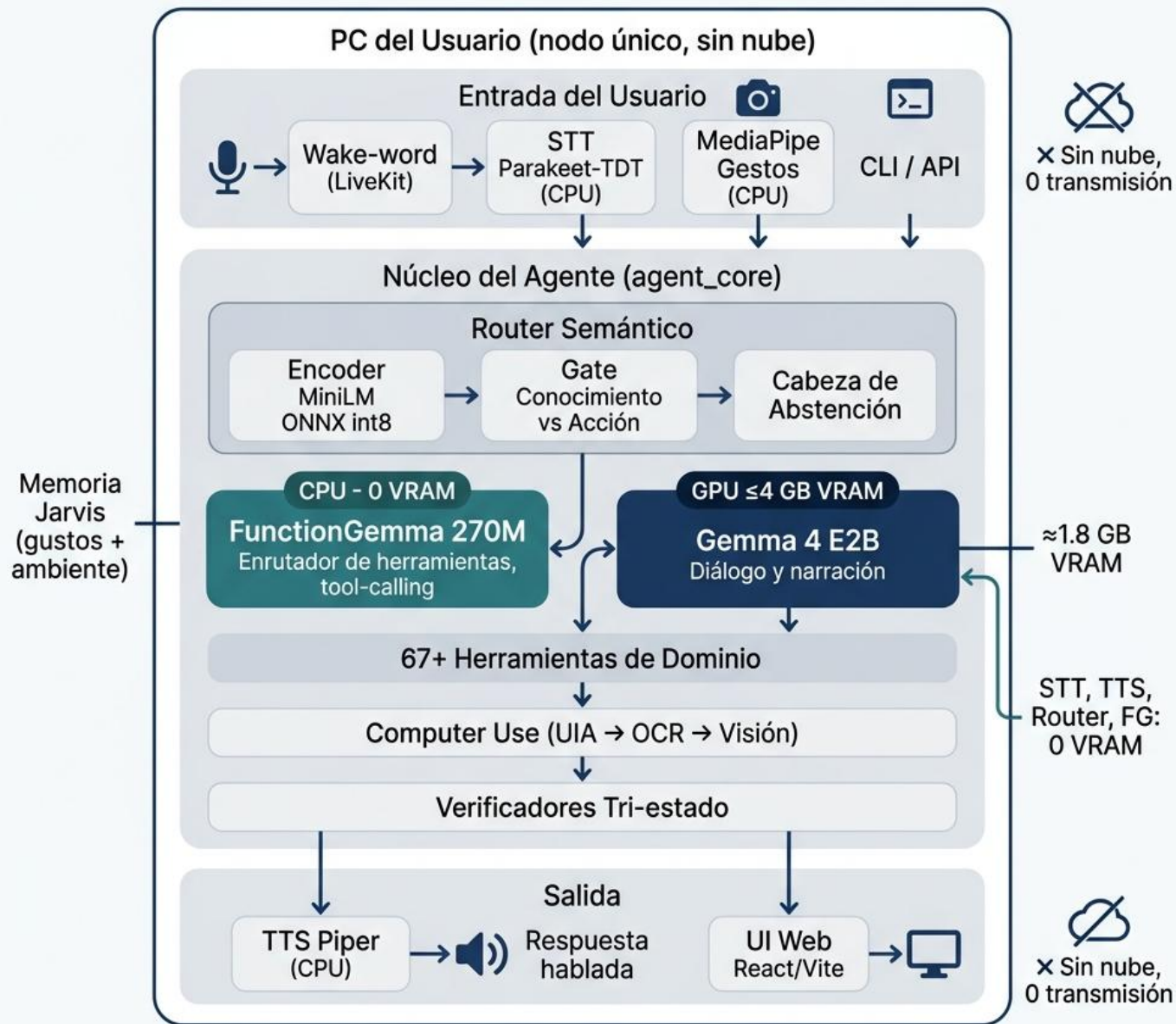


Movilidad

Manos libres: control total del PC solo por voz, sin teclado ni ratón.

**PROXIMAMENTE
DISPONIBLE**

Arquitectura General del Sistema Baxy — Arquitectura de Dos Modelos (Split)



Capacidades técnicas

Stack tecnológico

- Python 3.10 · FastAPI + Uvicorn
- React 19 + Vite + TypeScript · pywebview
- llama.cpp (CUDA) · Playwright · uiautomation (UIA)

67 herramientas compuestas

- Sistema · audio/multimedia · archivos
- Apps · juegos/Steam · navegador (Playwright)
- WhatsApp/email · productividad · desarrollo

Memoria y conocimiento

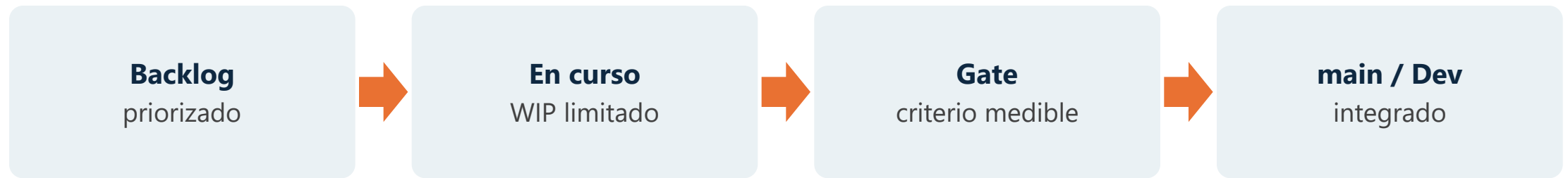
- RAG léxico: BM25 sobre SQLite FTS5 (+ rerank)
- Memoria multicapa (hechos, experience replay)
- Skills (spec Anthropic) · microagents (OpenHands)

Visión multimodal

- Visión nativa de Gemma + OCR (Tesseract)
- Lectura de pantalla para computer-use

Metodología ágil

Kanban — flujo continuo y WIP limitado, sin sprints de longitud fija



Cadencia

- Flujo continuo, sin sprints fijos
- Entregas diarias

Gate de aceptación

- Criterio medible, definido antes
- Recall · VRAM · latencia
- Validación en vivo antes de integrar

Definición de Hecho

- Tests verdes
- Gate medido
- Fuentes citadas
- Memoria actualizada

Matriz de riesgo

Riesgo	Impacto	Mitigación / contingencia
Alucinación del LLM	Alto	Verificación estructural de cada acción contra el SO + reintentos
Herramientas inventadas	Alto	Fine-tune QLoRA propio — gate de 0% en producción
Acciones irreversibles	Alto	Confirmación previa obligatoria + lógica anti-loop
GPU ocupada (juegos)	Medio	Watchdog: auto-fallback al perfil CPU (visión off)
Inestabilidad de Gemma	Medio	flash-attention off por default; perfiles VRAM (vram4 / cpu / standby)

Análisis legal y financiero

Marco legal y licencias

- Código propio bajo licencia MIT → permite uso comercial
- Modelo Gemma 4 bajo Gemma Terms of Use
- Dependencias open source (Whisper, Piper, Silero...) compatibles

Modelo de negocio

- Distribución dual: open source en GitHub (REDSOULTM/Baxy)
- Versión empaquetada de pago sobre la misma base de código
- Costo de operación ≈ 0 : 100% local, sin APIs de pago ni nube

Estado actual

MVP que excedió el alcance esperado — prueba de factibilidad técnica

~1.79 GB

VRAM real (perfil vram4_lean)

0.9521

Recall del routing (holdout)

0 %

Herramientas inventadas

67

Herramientas operativas

Ya funciona

- ✓ Voz local end-to-end: wake-word, STT y TTS en CPU
- ✓ Tool-Calling
- ✓ Triggers
- ✓ Operación de apps reales (cascada UIA → OCR → visión)
- ✓ Memoria multicapa en funcionamiento

¿De qué es capaz ahora?

1

Streaming

"pon Stranger Things en Netflix"

3

WhatsApp

"manda un whatsapp a "Migue" que diga ya voy"

5

Abrir apps / juegos

"abre la calculadora"

7

Reproducir Musica

"Pon Michael Jackson en Spotify"

9

Estado de la PC

"cuánta batería tengo"

2

Visión / leer pantalla

"describe lo que ves"

4

Buscar en internet

"busca el clima en Buenos Aires"

6

Control de media

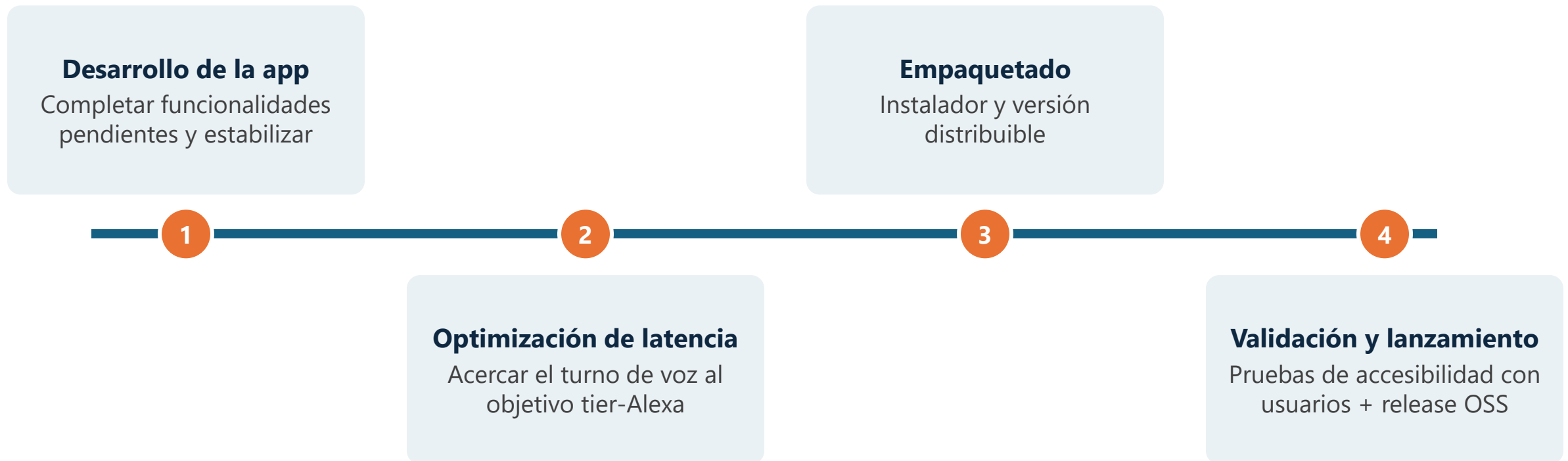
"sube el volumen"

8

Alarmas y recordatorios

"ponme una alarma a las 7"

Próximos pasos



EN DIRECTO · 5 MINUTOS

DEMO EN VIVO

Baxy funcionando — cómo cumple el objetivo general

Control por voz

Operar apps reales